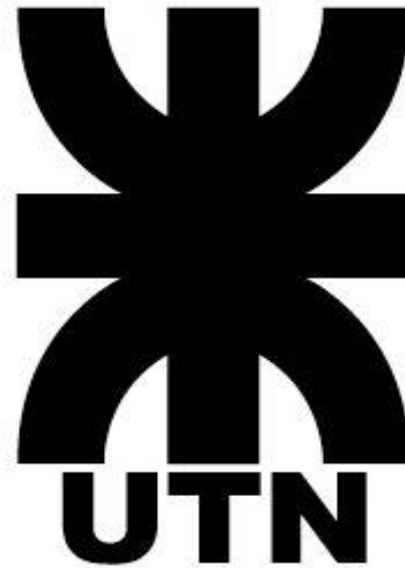


**UNIVERSIDAD TECNOLÓGICA  
NACIONAL  
FACULTAD REGIONAL DE TUCUMÁN**



**Actividad de Formación Práctica 2**

*Creación de drivers en STM32CubeIDE  
Programación de microcontroladores*

- **Materia:** Técnicas Digitales II
- **Comisión:** 4R1

**INTEGRANTES: GRUPO 1**

- Pistan Ulises
- Leguizamón Marcelo
- Rivero Martin
- Lucero Dario Alejandro

## • Contenido

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>Introducción .....</b>                                                             | <b>3</b>  |
| Utilización de drivers en el desarrollo de aplicaciones para Sistemas Embebidos ..... | 3         |
| <b>Creación de un driver en STM32CubeIDE .....</b>                                    | <b>4</b>  |
| API_GPIO.h .....                                                                      | 6         |
| API_GPIO.c .....                                                                      | 7         |
| <b>Implementación de los Drivers .....</b>                                            | <b>7</b>  |
| Aplicaciones desarrolladas por el grupo .....                                         | 9         |
| <b>Repositorio Grupal .....</b>                                                       | <b>12</b> |
| APP2.1 .....                                                                          | 12        |
| APP2.2 .....                                                                          | 12        |
| APP2.3 .....                                                                          | 12        |
| APP2.4 .....                                                                          | 12        |

# INTRODUCCIÓN

## Uso de Drivers en el Desarrollo de Aplicaciones para Sistemas Embebidos

En el ámbito del desarrollo de aplicaciones para sistemas embebidos, los *drivers* desempeñan un papel fundamental como puente entre el hardware y el software. Estos componentes de software se encargan de gestionar el funcionamiento de los dispositivos periféricos —como sensores, actuadores, pantallas, módulos de comunicación, entre otros— y permiten que el sistema operativo o la aplicación interactúe con ellos de manera estructurada, segura y eficiente.

### ¿Qué es un driver en un sistema embebido?

Un *driver* es un módulo de software diseñado específicamente para controlar un dispositivo hardware. Su función principal es traducir las instrucciones de alto nivel del software de aplicación a comandos que el hardware pueda entender y ejecutar. De esta manera, se aísla al programador de los detalles técnicos complejos y específicos del dispositivo.

### Ventajas del uso de drivers en sistemas embebidos

#### 1. Abstracción del hardware:

Los drivers ofrecen una capa de abstracción que oculta la complejidad de la comunicación directa con el hardware. Gracias a esto, los desarrolladores pueden trabajar con interfaces limpias y estandarizadas, sin preocuparse por registros, temporización, interrupciones u otros aspectos de bajo nivel.

#### 2. Modularidad y reutilización:

Al separar el control de hardware del resto de la lógica de la aplicación, se favorece un diseño modular. Esto permite que el mismo driver pueda ser reutilizado en diferentes proyectos, o que una aplicación se pueda portar fácilmente a un hardware distinto simplemente cambiando el driver correspondiente.

#### 3. Mantenimiento y escalabilidad:

Un sistema que utiliza drivers bien estructurados es más fácil de mantener y escalar. Si se detecta un fallo o se requiere una mejora, el cambio puede hacerse directamente en el driver, sin alterar la lógica principal del programa.

#### 4. Facilitación de pruebas y depuración:

Tener capas de software bien definidas ayuda a aislar los errores. Al utilizar drivers, los problemas relacionados con hardware pueden identificarse rápidamente sin necesidad de revisar el código de aplicación. Esto reduce el tiempo de desarrollo y facilita el trabajo en equipo.

#### 5. Portabilidad:

Los drivers permiten que el mismo software de aplicación funcione en diferentes plataformas hardware, siempre y cuando exista un driver compatible. Esto es esencial en entornos industriales donde se requiere flexibilidad frente a cambios tecnológicos.

## Consideraciones al desarrollar drivers

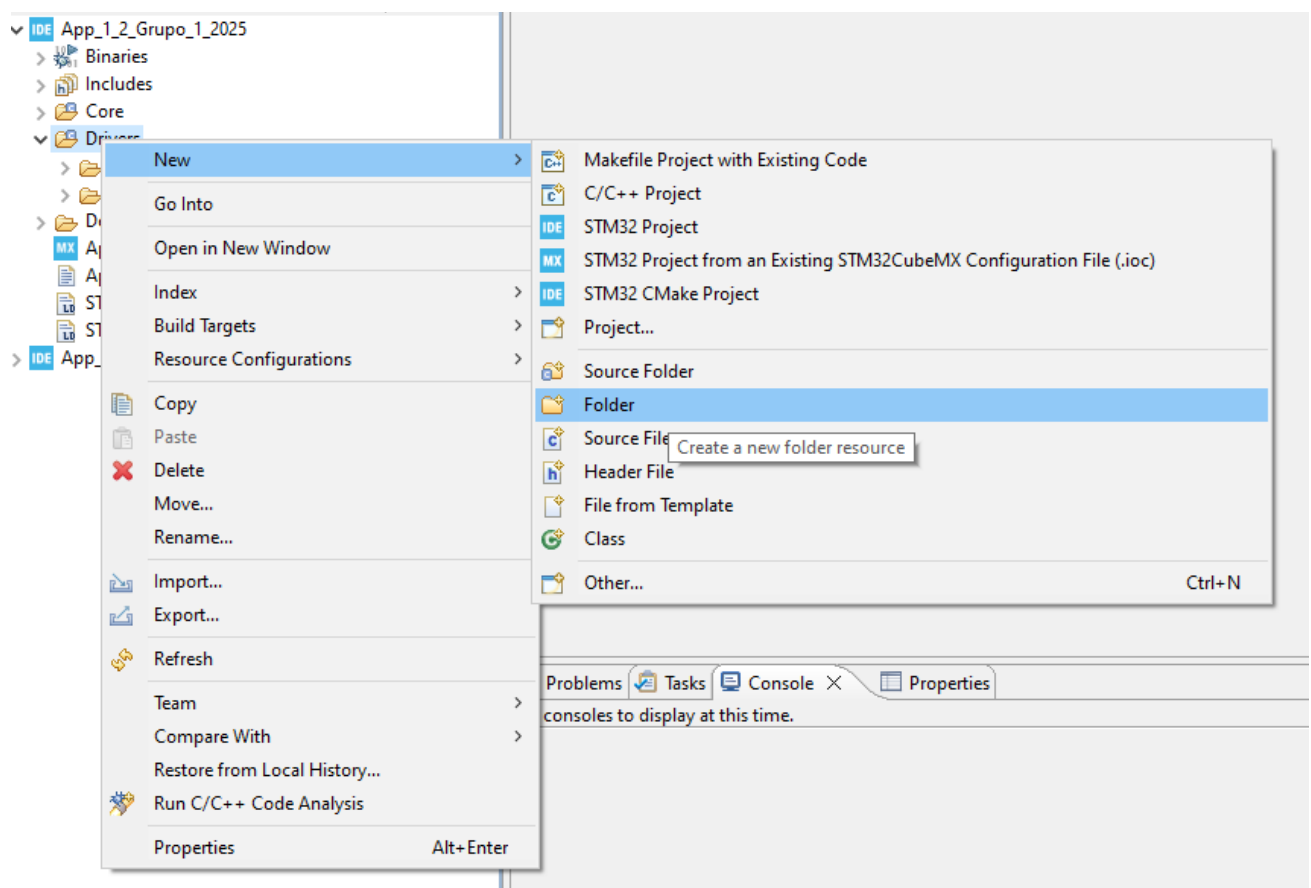
El desarrollo de drivers para sistemas embebidos requiere conocimientos detallados del hardware, manejo eficiente de recursos del sistema (memoria, CPU, interrupciones) y una comprensión clara del sistema operativo o entorno de ejecución (bare-metal, RTOS, Linux embebido, etc.). También es clave documentar correctamente cada driver para facilitar su integración y mantenimiento futuro.

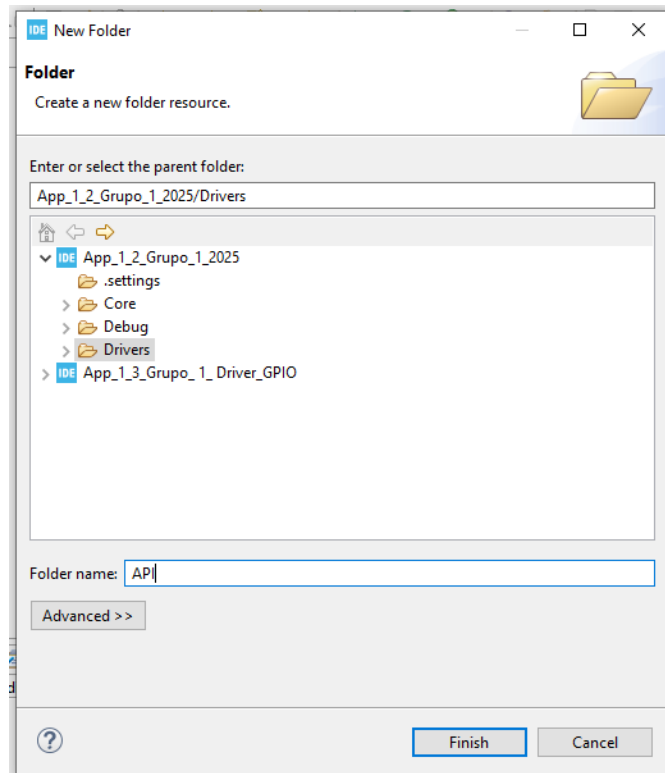
## Conclusión

El uso de drivers en sistemas embebidos no solo simplifica el desarrollo de aplicaciones complejas, sino que también mejora la calidad del software, facilita el mantenimiento, y promueve la reutilización y escalabilidad. Dominar la creación y utilización de drivers es una competencia esencial para cualquier ingeniero o desarrollador que trabaje en el campo de los sistemas embebidos.

## Creación de un driver en STM32CubeIDE

Lo primero que debemos hacer es crear una carpeta en donde vamos a crear la librería. Es recomendable crearlo dentro del directorio “./Drivers”. Esto se hace para mantener la organización del proyecto.

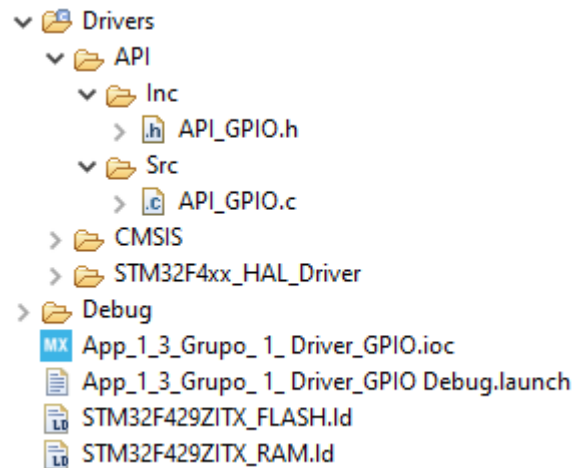




Dentro de esta carpeta “API” creamos dos subcarpetas:

- **Src:** La cual va a contener los archivos de extensión “.c” en donde se implementarán las funciones de dicha librería.
- **Inc:** En donde se guardarán los archivos de extensión “.h” conocidos como encabezados, en estos archivos se encuentran las declaraciones de las funciones, de las estructuras de tipo definido, los “#include” de las librerías necesarias y las constantes y etiquetas “define”.

El paso siguiente es crear dichos archivos.



## API\_GPIO.h

```
*API_GPIO.h X
1 /*
2  * API_GPIO.h
3  *
4  * Created on: Jul 24, 2025
5  * Author: GRUPO 1_2025:
6  *         LUCERO DARIO
7  *         PISTAN ULISES
8  *         LEGUIZAMON MARCELO
9  *         RIVERO MARTIN
10 */
11 #include <stdint.h>
12 #include <stdbool.h>
13
14 //Exportación de funciones prototipos
15
16 void MX_GPIO_Init(void);
17
18 // variables para transportar datos
19
20 typedef uint16_t led_t;
21 typedef bool buttonStatus_t;
22
23 //Funciones para modificar los estados de los leds
24
25 void writeLedOn_GPIO(led_t LDx);
26 void writeLedOff_GPIO(led_t LDx);
27 void toggleLed_GPIO(led_t LDx);
28
29 buttonStatus_t readButton_GPIO(void);
30
```

## API\_GPIO.c

```
*API_GPIO.h  *API_GPIO.c X
1  /*
2  *  API_GPIO.c
3  *
4  *   Created on: Jul 24, 2025
5  *   Author: GRUPO 1_2025:
6  *           LUCERO DARIO
7  *           PISTAN ULISES
8  *           LEGUIZAMON MARCELO
9  *           RIVERO MARTIN
10 /*
11 #include "main.h"
12
13 #include "API_GPIO.h"
14
15 led_t LDx;
16
17 //Definición de funciones
18
19 /**
20 * @brief GPIO Initialization Function
21 * @param None
22 * @retval None
23 */
24 void MX_GPIO_Init(void)
25 {
26     GPIO_InitTypeDef GPIO_InitStructure = {0};
27     /* USER CODE BEGIN MX_GPIO_Init_1 */
28
29     /* USER CODE END MX_GPIO_Init_1 */
30 }
```

## Implementación de los Drivers

Para implementar la librería en nuestra aplicación primero debemos escribir una línea de código:

**>> #include "API\_GPIO.h"**

Esto permite al archivo main.c acceder a las declaraciones de funciones expresadas en el ".h". Con lo cual es posible implementar las funciones de la librería en la aplicación principal.

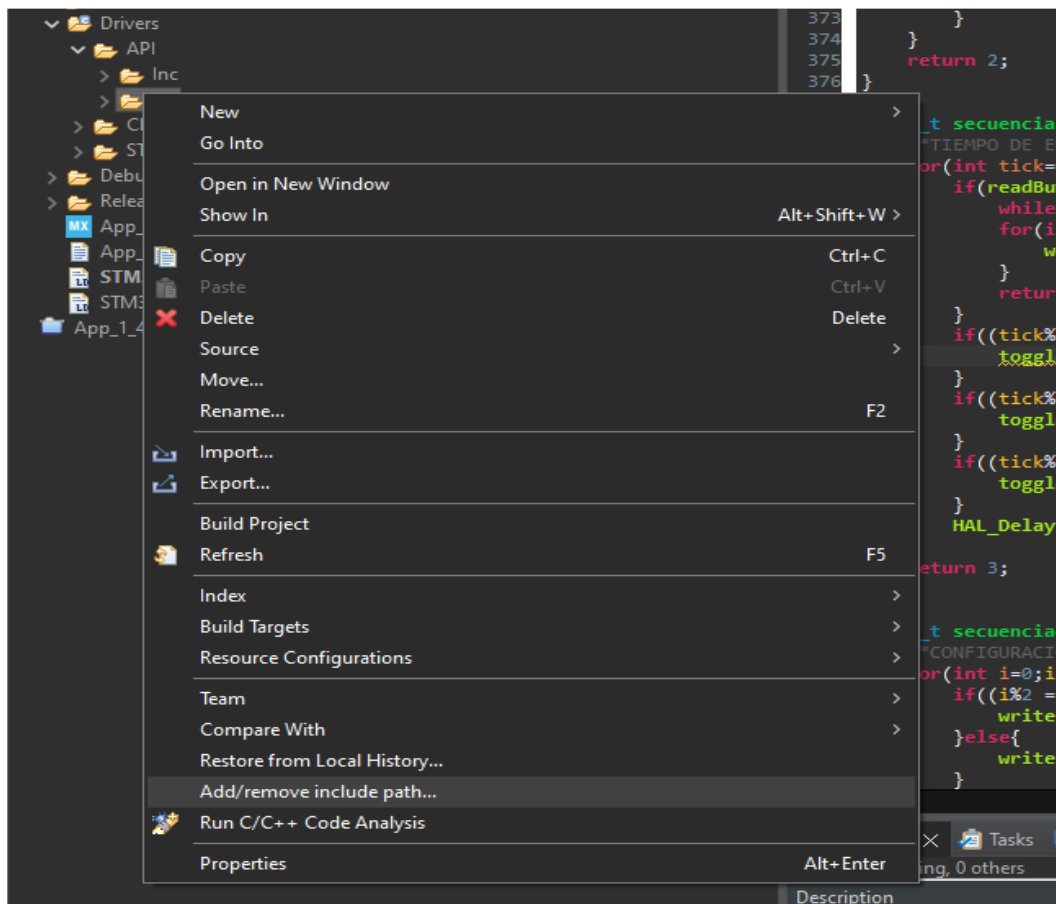
```
22 /* Private includes ----- */
23 /* USER CODE BEGIN Includes */
24 #include "API_GPIO.h"
```

Si intentamos compilar el proyecto en este punto obtenemos estos errores:

| Problems X Tasks Console Properties               |                |                      |          |                |  |
|---------------------------------------------------|----------------|----------------------|----------|----------------|--|
| 3 errors, 0 warnings, 0 others                    |                |                      |          |                |  |
| Description                                       | Resource       | Path                 | Location | Type           |  |
| v Errors (3 items)                                |                |                      |          |                |  |
| fatal error: API_GPIO.h: No such file or director | main.c         | /App_1_2_Grupo_1_... | line 25  | C/C++ Probl... |  |
| make: *** [Core/Src/subdir.mk:34: Core/Src/m      | App_1_2_Gru... |                      |          | C/C++ Probl... |  |
| make: *** Waiting for unfinished jobs....         | App_1_2_Gru... |                      |          | C/C++ Probl... |  |

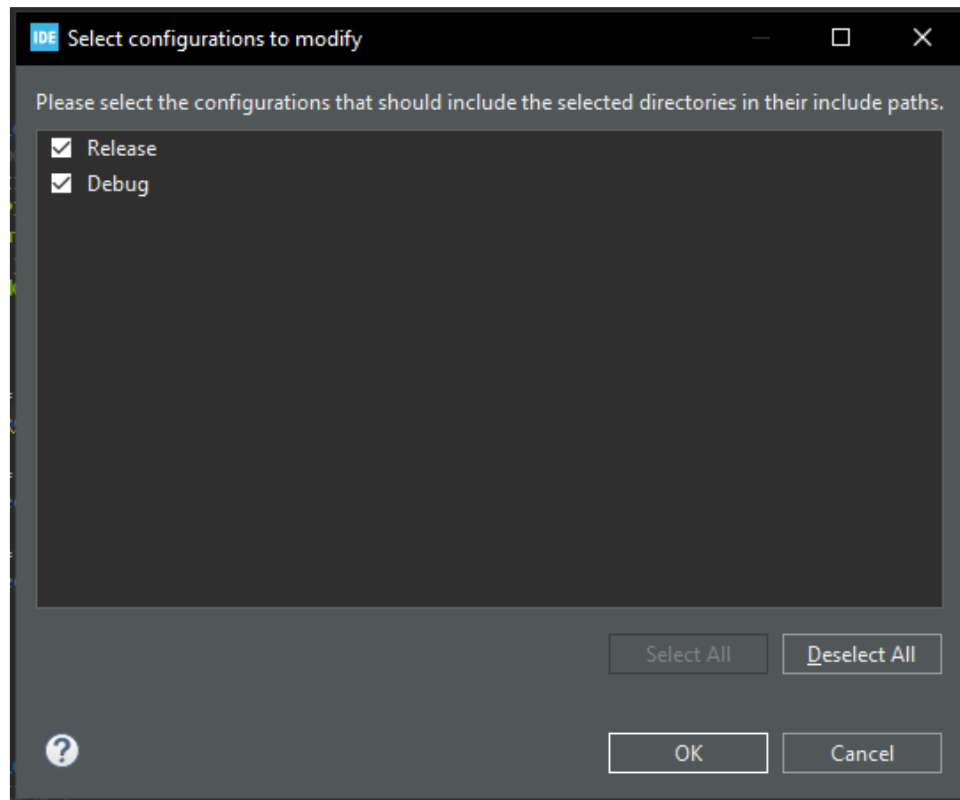
Esto se debe a que es necesario también agregar la librería a los directorios del proyecto. para que el compilador pueda incluirla en la compilación.

Nos dirigimos a cada una de las carpetas relacionadas a la librería y presionamos “Add/remove include path...”





En este menú nos aseguramos de que ambas opciones están tildadas y aceptamos.



Ahora la compilación se realizará con éxito.

Para implementar las funciones en el main.c, procedemos a cambiar el uso de las funciones HAL, que usamos inicialmente, por las nuevas funciones creadas en la librería. Es importante en este punto prestar atención a los parámetros que pasamos a las funciones.

Como ejemplo se puede tomar esta función, creada en el driver:

```
82 // Funciones para encender led
83 void writeLedOn_GPIO(led_t LDx)
84 {
85     HAL_GPIO_WritePin(GPIOB, LDx, GPIO_PIN_SET);
86 }
87
88 //Función para apagar led
89 void writeLedOff_GPIO(led_t LDx)
90 {
91     HAL_GPIO_WritePin(GPIOB, LDx, GPIO_PIN_RESET);
92 }
93
94 //funcion que se comporta como un inversor del estado del led
95 void toggleLed_GPIO(led_t LDx)
96 {
97     HAL_GPIO_TogglePin(GPIOB, LDx);
98 }
99
100 //funcion para la lectura de botones
101 buttonStatus_t readButton_GPIO(void)
102 {
103 }
```

Para intercambiar las siguientes líneas de código:

```
//*****  
//FUNCIONES DE SECUENCIA  
  
void Secuencia1(uint16_t VECTOR_Led[3])  
{  
    int retardo1=150 ;  
  
    for (uint8_t i = 0; i < 3; i++){  
  
        writeLedOn_GPIO( VECTOR_Led[i]);  
        HAL_Delay(retardo1);  
        writeLedOff_GPIO(VECTOR_Led[i]);  
  
    }  
  
}  
  
void Secuencia2(uint16_t VECTOR_Led[3])  
{  
  
    int retardo2=300;  
  
    writeLedOn_GPIO( VECTOR_Led[0]);  
    writeLedOn_GPIO( VECTOR_Led[1]);  
    writeLedOn_GPIO( VECTOR_Led[2]);  
  
    HAL_Delay(retardo2);  
  
    writeLedOff_GPIO(VECTOR_Led[0]);  
    writeLedOff_GPIO(VECTOR_Led[1]);  
    writeLedOff_GPIO(VECTOR_Led[2]);  
  
    HAL_Delay(retardo2);  
  
    }  
}
```

Implementación del uso del drivers para lectura de botones

```
// Lectura del botón  
estado_actual_boton= readButton_GPIO();
```

Uno de los beneficios más destacados del uso de *drivers* en sistemas embebidos es la **alta portabilidad del código**. Esta portabilidad permite que una misma aplicación pueda ejecutarse sobre diferentes plataformas de hardware con un mínimo de modificaciones. Esto se logra gracias a que cada driver actúa como una capa de abstracción entre el hardware específico y la lógica de la aplicación.

Cuando se implementa un driver dedicado para un dispositivo o periférico particular (por ejemplo, un sensor de temperatura, un módulo de comunicación SPI o un actuador), **la aplicación principal no necesita conocer ni gestionar los detalles de bajo nivel** como registros, interrupciones o protocolos de comunicación. En consecuencia, si el hardware es reemplazado o actualizado por uno diferente (por ejemplo, se cambia un sensor I2C por otro con distinta dirección o configuración), **el único componente que requiere ajustes es el driver**, manteniendo intacta la lógica del programa principal.

Esta arquitectura no solo mejora la portabilidad, sino que también tiene efectos positivos en otros aspectos clave del desarrollo:

### Mejora de la legibilidad del código

Al encapsular la funcionalidad específica del hardware dentro del driver, el código de la aplicación se vuelve mucho más claro y conciso. En lugar de tener líneas complejas que acceden directamente a registros de hardware o configuran periféricos, el desarrollador trabaja con funciones intuitivas como `Display_WriteText()`, `Sensor_ReadValue()`, o `Motor_Stop()`, lo que facilita la comprensión del código incluso para quienes no están familiarizados con los detalles técnicos del hardware utilizado.

### Facilita el trabajo en equipo y la colaboración

Esta separación de responsabilidades permite que diferentes miembros de un equipo de desarrollo trabajen de forma paralela sin interferencias. Por ejemplo, mientras un programador se enfoca en implementar o adaptar el driver para un nuevo módulo de comunicación, otro puede continuar desarrollando o depurando la lógica de control de la aplicación, sin necesidad de esperar a que el primero termine.

### Mantenimiento y escalabilidad

La encapsulación también facilita el mantenimiento del sistema. Si se detecta un error relacionado con un dispositivo específico, este se puede aislar rápidamente dentro del driver correspondiente. Asimismo, si se decide ampliar el sistema (por ejemplo, agregar nuevos sensores o módulos), simplemente se integran nuevos drivers sin necesidad de reescribir el núcleo de la aplicación.

### Consistencia y reutilización

Un driver bien diseñado, con una interfaz definida y documentada, puede reutilizarse en múltiples proyectos. Esto no solo acelera el tiempo de desarrollo, sino que también mejora la calidad del software, ya que se utilizan módulos probados y confiables.

## Aplicaciones desarrolladas por el grupo

### Repositorio Grupal

Link: [https://github.com/Dariuss-ing-electronica/Grupo\\_1\\_TDII\\_2025.git](https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025.git)

### App 2.1

**Autor:** Marcelo Leguizamon

Link: [https://github.com/Dariuss-ing-electronica/Grupo\\_1\\_TDII\\_2025/tree/main/AFP\\_2\\_TDII\\_2025/AFP\\_2\\_TDII\\_2025/App\\_2\\_1\\_Grupo\\_1\\_2025](https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_2_TDII_2025/AFP_2_TDII_2025/App_2_1_Grupo_1_2025)

### App 2.2

**Autor:** Martin Rivero

Link: [https://github.com/Dariuss-ing-electronica/Grupo\\_1\\_TDII\\_2025/tree/main/AFP\\_2\\_TDII\\_2025/AFP\\_2\\_TDII\\_2025/App\\_2\\_2\\_Grupo\\_%201\\_2025](https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_2_TDII_2025/AFP_2_TDII_2025/App_2_2_Grupo_%201_2025)

### App 2.3

**Autor:** Dario Lucero

Link: [https://github.com/Dariuss-ing-electronica/Grupo\\_1\\_TDII\\_2025/tree/main/AFP\\_2\\_TDII\\_2025/AFP\\_2\\_TDII\\_2025/App\\_2\\_3\\_Grupo\\_%201\\_2025](https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_2_TDII_2025/AFP_2_TDII_2025/App_2_3_Grupo_%201_2025)

### App 2.4

**Autor:** Pistan Ulises

Link: [https://github.com/Dariuss-ing-electronica/Grupo\\_1\\_TDII\\_2025/tree/main/AFP\\_2\\_TDII\\_2025/AFP\\_2\\_TDII\\_2025/App\\_2\\_4\\_Grupo\\_1\\_2025/App\\_2\\_4\\_Grupo\\_1\\_2025](https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_2_TDII_2025/AFP_2_TDII_2025/App_2_4_Grupo_1_2025/App_2_4_Grupo_1_2025)